Static analysis and software verification

Lecture 4 - Dataflow analyses

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it

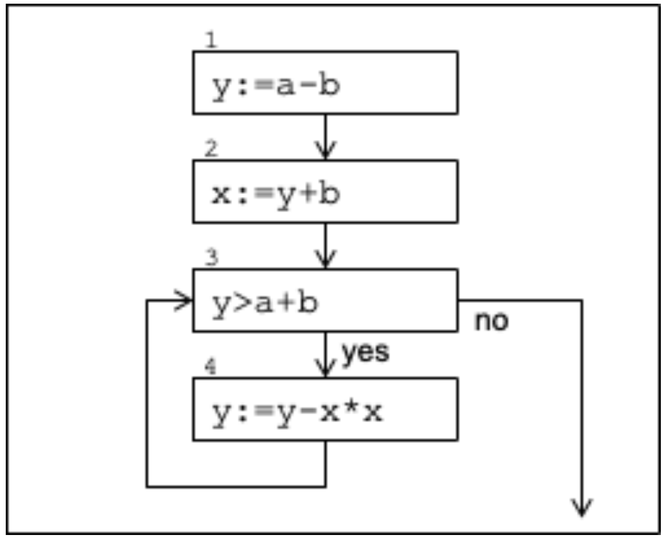
Control Flow Graph (CFG)

- In dataflow analysis (and other analyses) programs are typically represented as CFGs

Control Flow Graph (CFG)

- In dataflow analysis (and other analyses) programs are typically represented as CFGs

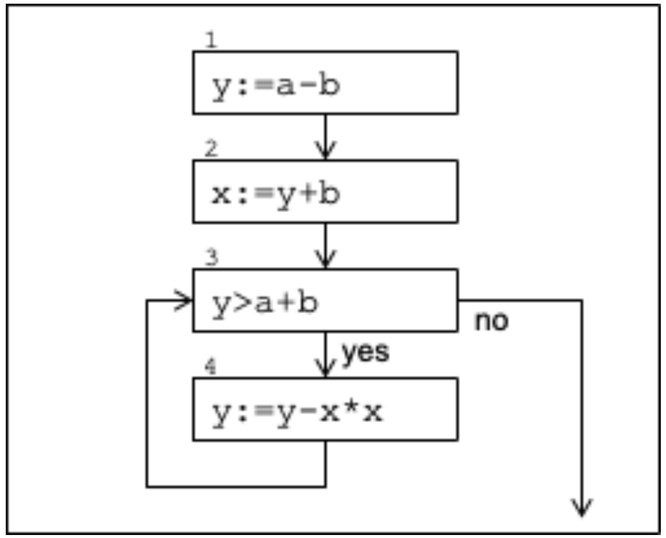
```
1 y = a - b;
2 x = y + b;
3 while (y > a + b)
4   y = y - x * x;
```



Control Flow Graph (CFG)

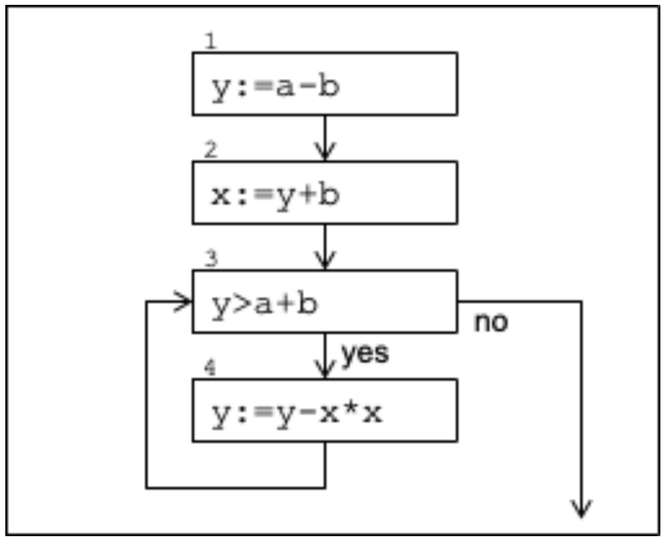
- In dataflow analysis (and other analyses) programs are typically represented as CFGs

```
1 y = a - b;
2 x = y + b;
3 while (y > a + b)
4   y = y - x * x;
```



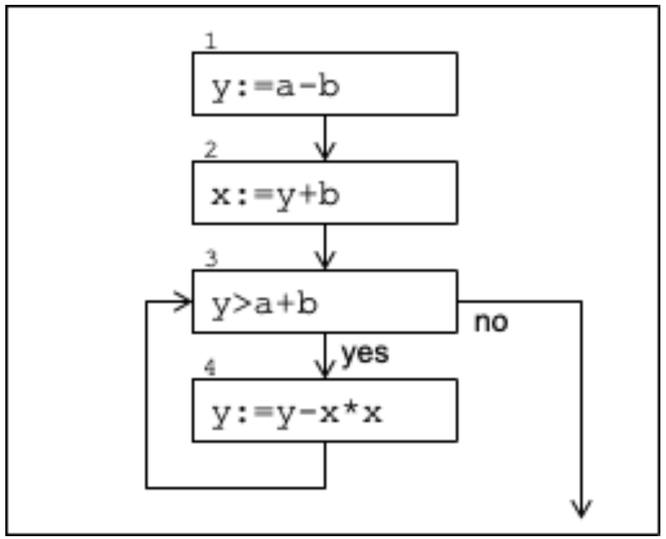
- Each statement is a labeled block
- Each block has one outgoing or two outgoing edges

Control Flow Graph (CFG)



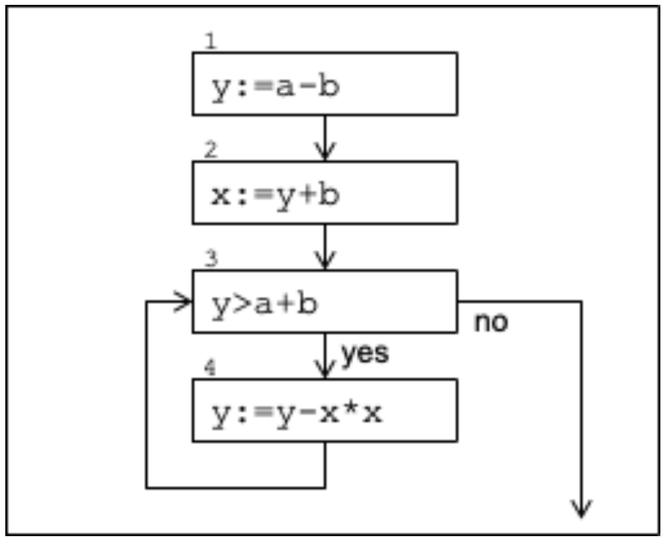
Control Flow Graph (CFG)

- Nodes are single statements, identified by $i \in \mathbb{N}$



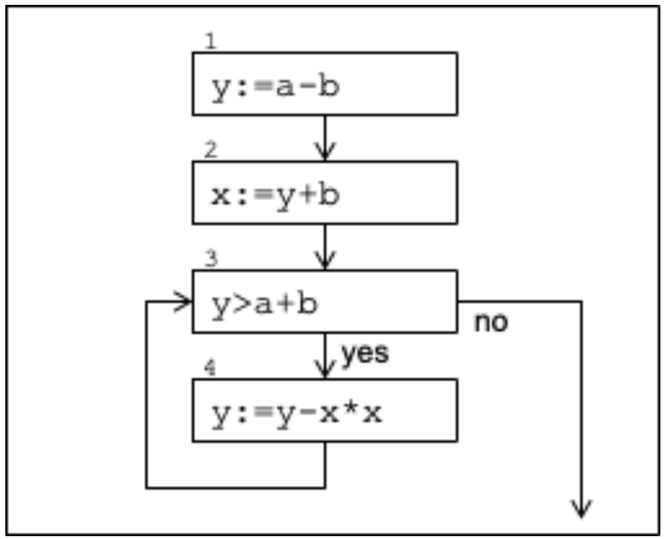
Control Flow Graph (CFG)

- Nodes are single statements, identified by $i \in \mathbb{N}$
- Edges are pairs $(i_1, i_2) \in \mathbb{N} \times \mathbb{N}$



Control Flow Graph (CFG)

- Nodes are single statements, identified by $i \in \mathbb{N}$
- Edges are pairs $(i_1, i_2) \in \mathbb{N} \times \mathbb{N}$
- Given a CFG
  - nodes(G): set of all indexes of blocks in G
  - stmt(G, i): the statement contained in the i block of G
  - edges(G) $\subseteq \mathbb{N} \times \mathbb{N}$: the edges



Dataflow analyses

Dataflow analyses

- Traditional technique for compiler optimizations

Dataflow analyses

- Traditional technique for compiler optimizations
- Programs are typically represented by CFGs

Dataflow analyses

- Traditional technique for compiler optimizations
- Programs are typically represented by CFGs
- Approach based on equations

Dataflow analyses

- Traditional technique for compiler optimizations
- Programs are typically represented by CFGs
- Approach based on equations
  - Relations between entry and exit states of each block

Dataflow analyses

- Traditional technique for compiler optimizations
- Programs are typically represented by CFGs
- Approach based on equations
  - Relations between entry and exit states of each block
  - Solve the equations (obtaining an over-approximation of the property of interest)

Dataflow analyses

- Traditional technique for compiler optimizations
- Programs are typically represented by CFGs
- Approach based on equations
  - Relations between entry and exit states of each block
  - Solve the equations (obtaining an over-approximation of the property of interest)
- The property of interest is usually simple and oriented to optimize runtime execution (e.g., avoid to compute twice the same expression)

Available expressions

"For each program point, which expressions myst have already been computed, and not later modified, on all path to the program point"

Available expressions

"For each program point, which expressions myst have already been computed, and not later modified, on all path to the program point"

- Useful to
  - avoid to recompute the same expression twice
  - store it in a temporary register
  - we focus on arithmetic expressions

Available expressions

"For each program point, which expressions must have already been computed, and not later modified, on all path to the program point"

Available expressions

"For each program point, which expressions must have already been computed, and not later modified, on all path to the program point"

- When x is assigned all expressions containing x are not available anymore (*killed)*

Available expressions

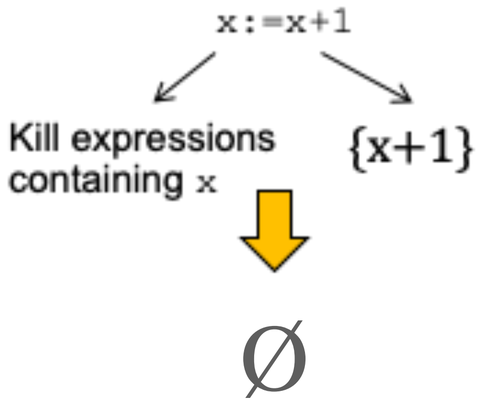
"For each program point, which expressions must have already been computed, and not later modified, on all path to the program point"

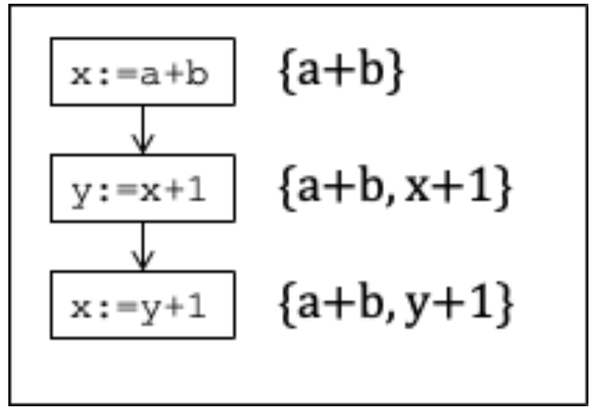
- When x is assigned all expressions containing x are not available anymore (*killed)*
- When x is assigned, all the expressions computed become available (*gen)*

Available expressions

Example

"For each program point, which expressions must have already been computed, and not later modified, on all path to the program point"





Available expressions

Formalization

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$
- AExp(e): non-trivial arithmetic expression in e

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$
- AExp(e): non-trivial arithmetic expression in e
- AExp(b): non-trivial arithmetic expression in b

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$
- AExp(e): non-trivial arithmetic expression in e
- AExp(b): non-trivial arithmetic expression in b
- FV(a): free-variables in a

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$
- AExp(e): non-trivial arithmetic expression in e
- AExp(b): non-trivial arithmetic expression in b
- FV(a): free-variables in a

$$\text{gen}(x := e) = \{a \in AExp(e) \mid x \notin \text{FV}(a)\}$$

Available expressions

Formalization

- For each program point (i.e., for each block) we track the set of available expressions
  - thus, the state of the analysis is an element of $\wp(AE)$
- AExp(e): non-trivial arithmetic expression in e
- AExp(b): non-trivial arithmetic expression in b
- FV(a): free-variables in a

$$\text{kill}(x := e) = \{a \in AExp(e) \mid x \in \text{FV}(a)\}$$

$$\text{gen}(x := e) = \{a \in AExp(e) \mid x \notin \text{FV}(a)\}$$

Available expressions

Entry and exit states

Available expressions

Entry and exit states

- Available expressions entering a block: the ones available after the previous block

Available expressions

Entry and exit states

- Available expressions entering a block: the ones available after the previous block
- Available expression analysis is a *forward* analysis, i.e., the information is propagated from entry to exit states

Available expressions

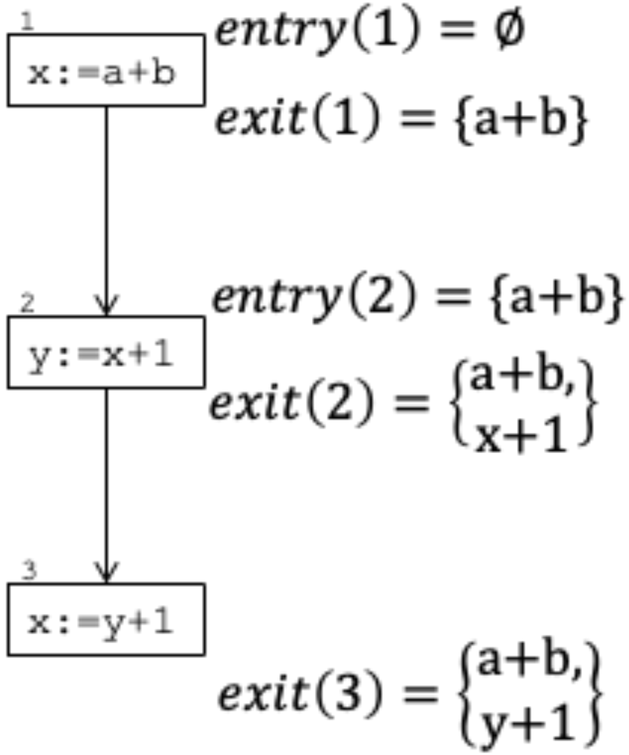
Entry and exit states

- Available expressions entering a block: the ones available after the previous block
- Available expression analysis is a *forward* analysis, i.e., the information is propagated from entry to exit states

$$\text{entry}(pp) = \text{exit}(\text{predecessor}(pp))$$

Available expressions

Example



Available expressions

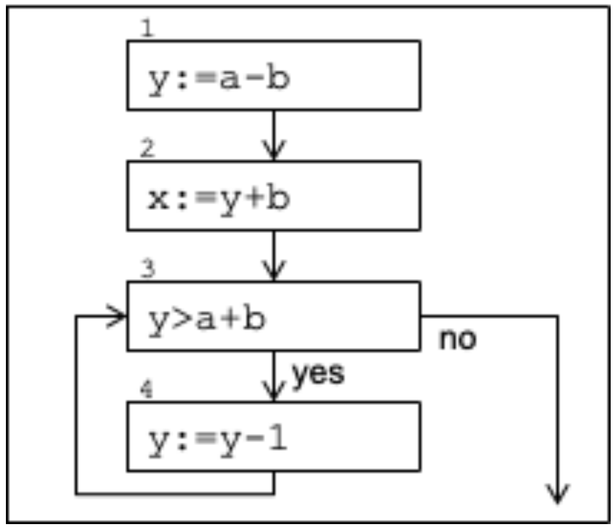
Join

"For each program point, which expressions must have already been computed, and not later modified, on all path to the program point"

Available expressions

Join

"For each program point, which expressions myst have already been computed, and not later modified, on all path to the program point"



exit$(2) = \{a - b, y + b\}$

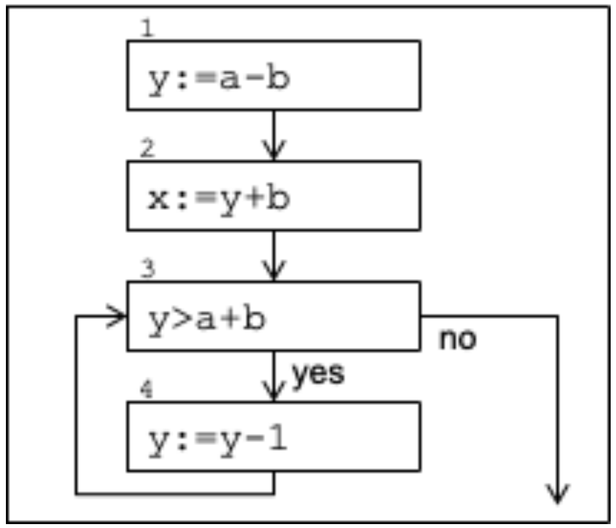
exit$(4) = \{a - b, a + b\}$

entry$(3) = ?$

Available expressions

Join

"For each program point, which expressions myst have already been computed, and not later modified, on all path to the program point"



exit(2) $= \{a - b, y + b\}$

exit(4) $= \{a - b, a + b\}$

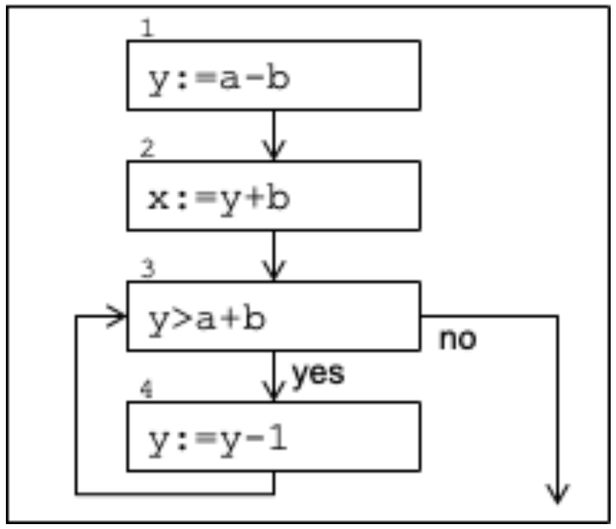
entry(3) $= ?$

Available expressions is a must analysis,
i.e., if something is available surely it is

Available expressions

Join

"For each program point, which expressions myst have already been computed, and not later modified, on all path to the program point"



$\text{exit}(2) = \{a - b, y + b\}$

$\text{exit}(4) = \{a - b, a + b\}$

$\text{entry}(3) = \text{exit}(2) \cap \text{exit}(4) = \{a - b\}$

Available expressions is a must analysis, i.e., if something is available surely it is

Available expressions

Formalization

Available expressions

Formalization

- Available expressions entering a block i

Available expressions

Formalization

- Available expressions entering a block i

$$\text{entry}(i) = \cap \{\text{exit}(i') \mid (i', i) \in \text{edges}(G)\}$$

Available expressions

Formalization

- Available expressions entering a block i

$$\text{entry}(i) = \cap \{\text{exit}(i') \mid (i', i) \in \text{edges}(G)\}$$

- Available expressions exiting a block i

Available expressions

Formalization

- Available expressions entering a block i

$$\text{entry}(i) = \cap \{\text{exit}(i') \mid (i', i) \in \text{edges}(G)\}$$

- Available expressions exiting a block i

$$\text{exit}(i) = (\text{entry}(i) \setminus \text{kill}(\text{stmt}(G, i))) \cup \text{gen}(\text{stmt}(G, i))$$

Available expressions

Formalization

- Available expressions entering a block i

$$\text{entry}(i) = \cap \{\text{exit}(i') \mid (i', i) \in \text{edges}(G)\}$$

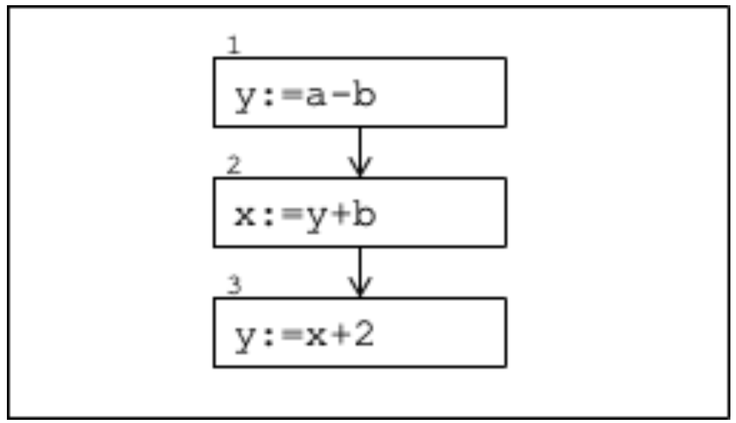
- Available expressions exiting a block i

$$\text{exit}(i) = (\text{entry}(i) \setminus \text{kill}(\text{stmt}(G, i))) \cup \text{gen}(\text{stmt}(G, i))$$

- Now, iterations to compute them (until a fixpoint is reached)

Available expressions

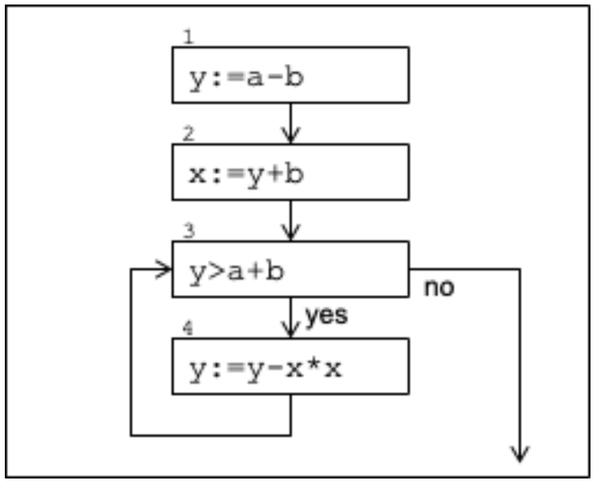
Example



Block	entry	exit
1	$\varnothing$	$\big(\text{entry}(1) \setminus \{a \in AE \mid y \in FV(a)\}\big) \cup \{a-b\}$
2	$\text{exit}(1)$	$\big(\text{entry}(2) \setminus \{a \in AE \mid x \in FV(a)\}\big) \cup \{y+b\}$
3	$\text{exit}(2)$	$\big(\text{entry}(3) \setminus \{a \in AE \mid y \in FV(a)\}\big) \cup \{x+2\}$

Available expressions

Example



Block	entry	exit
1	$\varnothing$	$\big(\mathsf{entry}(1) \setminus \mathsf{kill}(y := a - b)\big) \cup \mathsf{gen}(y := a - b)$
2	$\mathsf{exit}(1)$	$\big(\mathsf{entry}(2) \setminus \mathsf{kill}(x := y + b)\big) \cup \mathsf{gen}(x := y + b)$
3	$\mathsf{exit}(2) \cap \mathsf{exit}(4)$	$\big(\mathsf{entry}(3) \setminus \mathsf{kill}(y > a + b)\big) \cup \mathsf{gen}(y > a + b)$
4	$\mathsf{exit}(3)$	$\big(\mathsf{entry}(4) \setminus \mathsf{kill}(y := y - x * x)\big) \cup \mathsf{gen}(y := y - x * x)$

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

we know nothing at the beginning

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

we know nothing at the beginning

no expression is available at the entry node

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

we know nothing at the beginning

no expression is available at the entry node

changed contains the nodes to be evaluated

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

we know nothing at the beginning

no expression is available at the entry node

changed contains the nodes to be evaluated

compute the entry state

compute the exit state

Available expressions

```
1:  for all i ∈ nodes(G) do
2:      exit(i) ← AE
3:  end for
4:  entry(1) ← ∅
5:  exit(1) ← (entry(1) \ kill(stmt(G, 1))) ∪ gen(stmt(G, 1))
6:  changed ← nodes(cfg) \ {1}
7:  while changed ≠ ∅ do
8:      Pick i ∈ changed
9:      changed ← changed \ {i}
10:     entry(i) ← AE
11:     for all p such that (p, i) ∈ edges(G) do
12:         entry(i) ← entry(i) ∩ exit(p)
13:     end for
14:     exit(i) ← (entry(i) \ kill(stmt(G, i))) ∪ gen(stmt(cfg, i))
15:     if exit(i) changed then
16:         for all p such that (i, p) ∈ edges(G) do
17:             changed ← changed ∪ {p}
18:         end for
19:     end if
20: end while
```

we know nothing at the beginning

no expression is available at the entry node

changed contains the nodes to be evaluated

compute the entry state

compute the exit state

update the set of nodes that have to be evaluated again

Other dataflow analyses

From compiler course

Other dataflow analyses

From compiler course

Live variables: *"For each program point, which variables may be live at the exit from the program point. A variable is live if its content will be read in some paths starting from the program point."*

Other dataflow analyses

From compiler course

Live variables: *"For each program point, which variables may be live at the exit from the program point. A variable is live if its content will be read in some paths starting from the program point."*

Reaching definitions: *"For each program point, which assignments may have been made and not overwritten, when program execution reaches this point along some path."*

Other dataflow analyses

From compiler course

Live variables: *"For each program point, which variables may be live at the exit from the program point. A variable is live if its content will be read in some paths starting from the program point."*

Reaching definitions: *"For each program point, which assignments may have been made and not overwritten, when program execution reaches this point along some path."*

Very busy expressions: *"For each program point, which expressions must be very busy at the exit from the point. An expression is very busy if it will be always used in all paths starting from a label before any of the variables occurring in it are redefined."*

Other dataflow analyses

From compiler course

	Possible	Definite
Foward	**Reaching definitions** *"For each program point, which assignments **may** have been made and not overwritten, when program execution reaches this point along **some path**."*	**Available expressions** *"For each program point, which expressions **must** have already been computed, and not later modified, on **all paths** to the program point"*
Backword	**Live variables** *"For each program point, which variables **may** be live at the exit from the program point. A variable is live if its content will be read in **some path** starting from the program point."*	**Very busy expressions** *"For each program point, which expressions **must** be very busy at the exit from the point. An expression is very busy if it will be always used in **all paths** starting from a label before any of the variables occurring in it are redefined."*

Other dataflow analyses

From compiler course

	Possible	Definite
Foward	**Reaching definitions** *"For each program point, which assignments **may** have been made and not overwritten, when program execution reaches this point along **some path**."*	**Available expressions** *"For each program point, which expressions **must** have already been computed, and not later modified, on **all paths** to the program point"*
Backword	**Live variables** *"For each program point, which variables **may** be live at the exit from the program point. A variable is live if its content will be read in **some path** starting from the program point."*	**Very busy expressions** *"For each program point, which expressions **must** be very busy at the exit from the point. An expression is very busy if it will be always used in **all paths** starting from a label before any of the variables occurring in it are redefined."*

- Possible: may, some paths
- Definite: must, all paths

Other dataflow analyses

From compiler course

	Possible	Definite
Foward	**Reaching definitions** *"For each program point, which assignments **may** have been made and not overwritten, when program execution reaches this point along **some path**."*	**Available expressions** *"For each program point, which expressions **must** have already been computed, and not later modified, on **all paths** to the program point"*
Backword	**Live variables** *"For each program point, which variables **may** be live at the exit from the program point. A variable is live if its content will be read in **some path** starting from the program point."*	**Very busy expressions** *"For each program point, which expressions **must** be very busy at the exit from the point. An expression is very busy if it will be always used in **all paths** starting from a label before any of the variables occurring in it are redefined."*

- Possible: may, some paths
- Definite: must, all paths
- Forward: have been, to the pp
- Backward: exit, from the pp

Other dataflow analyses

From compiler course

	Possible	Definite
Foward	**Reaching definitions** *"For each program point, which assignments **may** have been made and not overwritten, when program execution reaches this point along **some path**."*	**Available expressions** *"For each program point, which expressions **must** have already been computed, and not later modified, on **all paths** to the program point"*
Backword	**Live variables** *"For each program point, which variables **may** be live at the exit from the program point. A variable is live if its content will be read in **some path** starting from the program point."*	**Very busy expressions** *"For each program point, which expressions **must** be very busy at the exit from the point. An expression is very busy if it will be always used in **all paths** starting from a label before any of the variables occurring in it are redefined."*

- It is possible to define a generic algorithm parametric on kill, gen, how states are merged at join point, etc… (see compiler course, monotone framework)

Lattices in dataflow analyses

- Available expressions: $\langle \wp(AE), \supseteq, AE, \emptyset, \cap, \cup \rangle$
- Live variables: $\langle \wp(Var), \subseteq, \emptyset, Var, \cup, \cap \rangle$
- Reaching definitions: …
- Very busy expressions: …

Lattices in dataflow analyses

- Available expressions: $\langle \wp(AE), \supseteq, AE, \emptyset, \cap, \cup \rangle$
- Live variables: $\langle \wp(Var), \subseteq, \emptyset, Var, \cup, \cap \rangle$
- Reaching definitions: …
- Very busy expressions: …
- All dataflow analyses are complete lattices
- We can prove correctness and termination (out-of-scope, see compiler course)
- Can we model all static analyses as dataflow analyses?

Lattices in dataflow analyses

- Available expressions: $\langle \wp(AE), \supseteq, AE, \emptyset, \cap, \cup \rangle$
- Live variables: $\langle \wp(Var), \subseteq, \emptyset, Var, \cup, \cap \rangle$
- Reaching definitions: …
- Very busy expressions: …
- All dataflow analyses are complete lattices (and must satisfy ACC)
- We can prove correctness and termination (out-of-scope, see compiler course)
- Can we model all static analyses as dataflow analyses? No

Distributive analyses

- $\langle \wp(X), \subseteq, \emptyset, X, \cup, \cap \rangle$ and $f : \wp(X) \to \wp(X)$
- f is distributive is $f(x \cup y) = f(x) \cup f(y)$
- All dataflow analyses are distributive analysis (even when lub is $\cap$)

Distributive analyses

Distributive analyses

- Distributive analyses are *easy*

Distributive analyses

- Distributive analyses are *easy*
- Track *how* the computation is performed

Distributive analyses

- Distributive analyses are *easy*
- Track *how* the computation is performed
  - variables that are assigned

Distributive analyses

- Distributive analyses are *easy*
- Track *how* the computation is performed
  - variables that are assigned
  - expressions that are computed

Distributive analyses

- Distributive analyses are *easy*
- Track *how* the computation is performed
  - variables that are assigned
  - expressions that are computed
- Not *what* the program computes

Distributive analyses

- Distributive analyses are *easy*
- Track *how* the computation is performed
  - variables that are assigned
  - expressions that are computed
- Not *what* the program computes
- All "difficult" analyses are not distributive, and often cannot be formalized as dataflow analyses

Distributive analyses

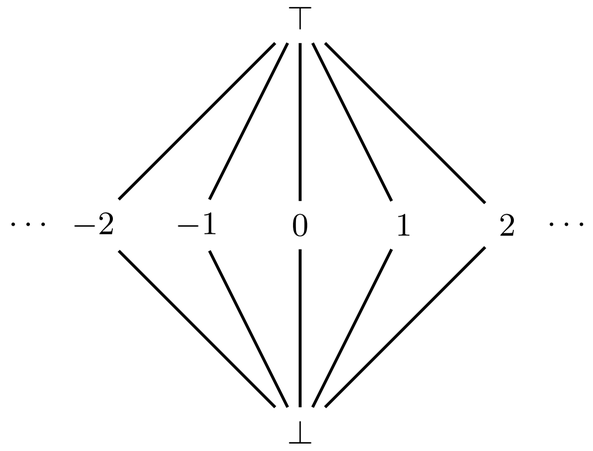
- Distributive analyses are *easy*
- Track *how* the computation is performed
  - variables that are assigned
  - expressions that are computed
- Not *what* the program computes
- All "difficult" analyses are not distributive, and often cannot be formalized as dataflow analyses
- That's why abstract interpretation has been introduced

Constant propagation

"For each program point, whether or not a variable has a constant value whenever an execution reaches that point.

- Constant propagation is not a distributive analysis

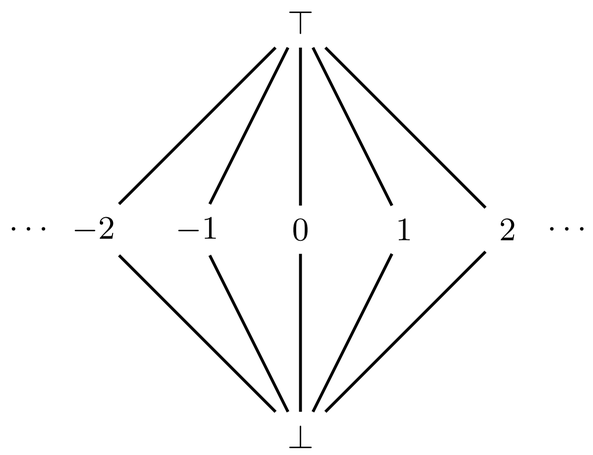
$$CP = \langle \{ \bot \} \cup \mathbb{Z} \cup \{ \top \}, \sqsubseteq, \sqcup, \sqcap, \bot, \top \rangle$$

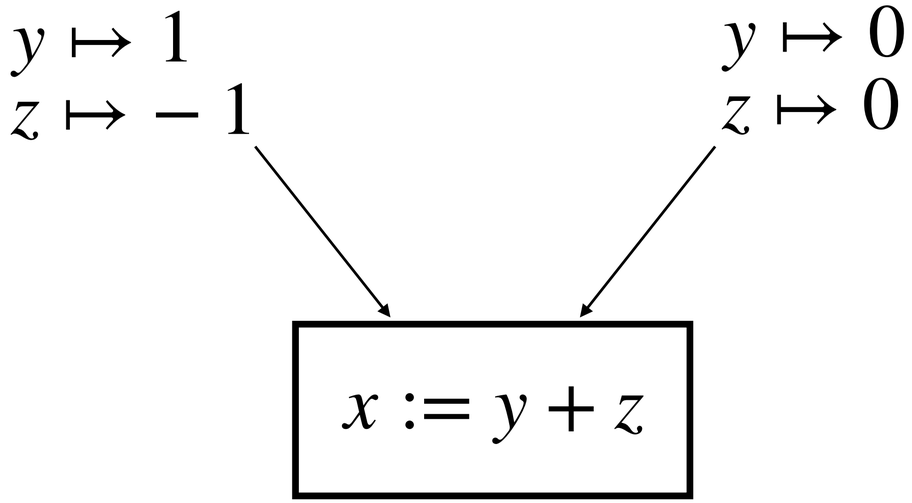


Constant propagation

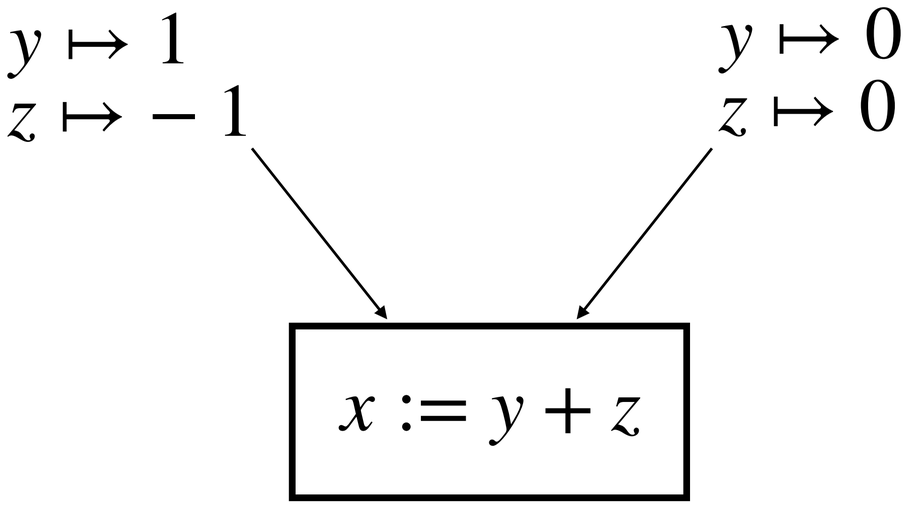
"For each program point, whether or not a variable has a constant value whenever an execution reaches that point.

$\mathsf{CP} = \langle \{ \bot \} \cup \mathbb{Z} \cup \{ \top \}, \sqsubseteq, \sqcup, \sqcap, \bot, \top \rangle$

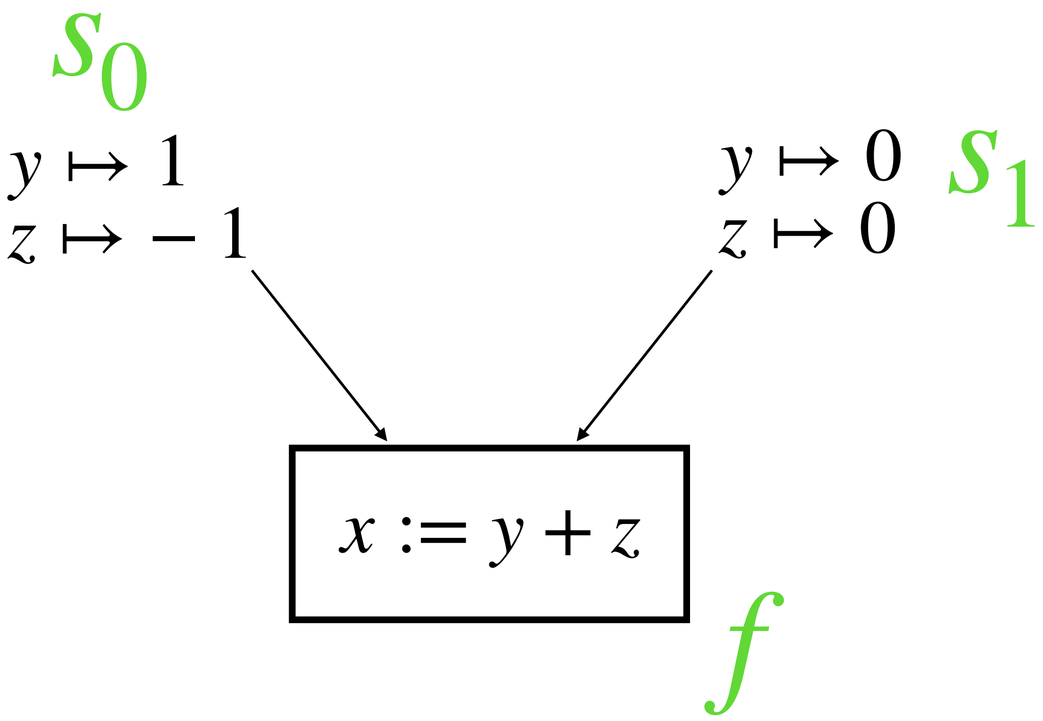




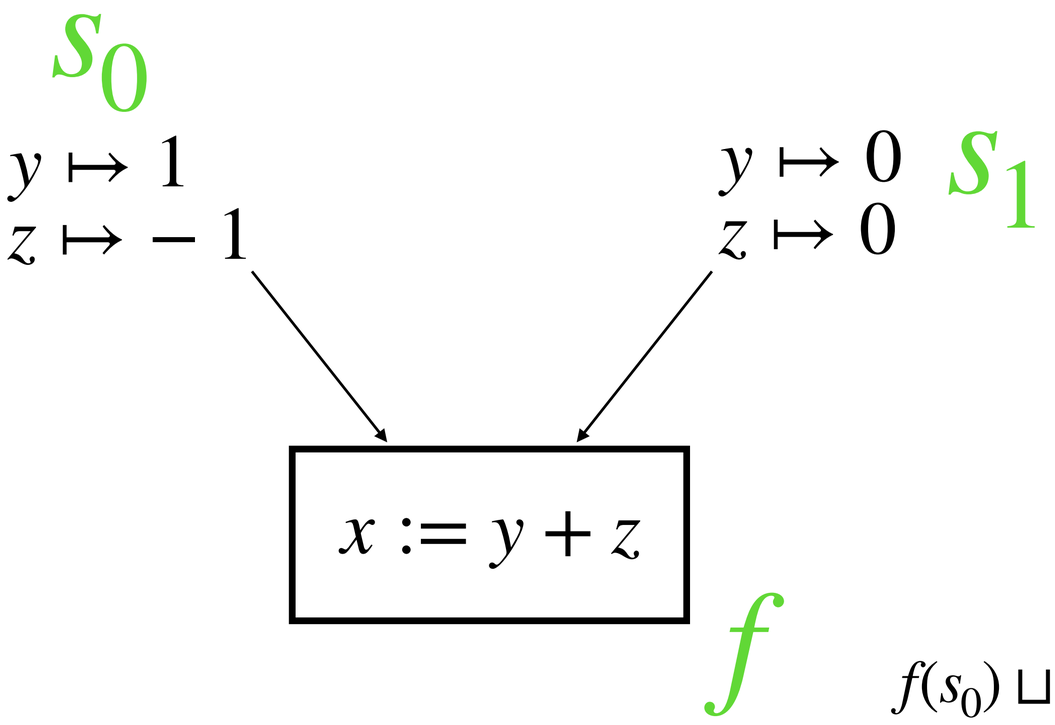
Constant propagation



Constant propagation



Constant propagation



- If constant propagation was distributive the following fact would hold

$$f(s_0 \sqcup s_1) = f(s_0) \sqcup f(s_1)$$

- But

$$f(s_0 \sqcup s_1) = [x \mapsto \top, y \mapsto \top, z \mapsto \top]$$

$$f(s_0) \sqcup f(s_1) = [x \mapsto 0, y \mapsto 1, z \mapsto -1] \sqcup [x \mapsto 0, y \mapsto 0, z \mapsto 0]$$
$$= [x \mapsto 0, y \mapsto \top, z \mapsto \top]$$

Distributive analyses

Example

- Is interval analysis a distribute analysis?